1. Protocol

Curve Llamalend

2. General functioning

Curve Llamalend is Curve's lending protocol. It consists of one way and isolated overcollateralised lending markets, where users either borrow crvUSD, Curve's stable coin, against collateral tokens, or borrow tokens against crvUSD (crvUSD must be either the collateral or the coin being borrowed).
Like other lending protocols, it allows lenders to generate interests, and borrowers to manage their exposure to different assets.

What sets it apart is the soft liquidation mechanism, where liquidation doesn't happen all at once but gradually within a range, solving problems caused by hard liquidation like bad debt, liquidation cascades, and improving users risk management.

Each market has its own Controller, the interface for most user actions, its own LLAMMA, the AMM that holds the collateral assets and handles soft liquidation, and its own ERC4626 Vault, where users provide the market with assets to be borrowed, assets that the vault then deposits in the Controller, and tracks the fees earned by lenders. Lenders obtain supply vault shares, that increase in value as the interests are collected.

Anyone can create a lending market provided they use a valid price oracle, and crvUSD is one of the tokens.

Lenders can stake their supply vault shares into the lending market's reward gauge, if one exists (this depends on the governance of Curve and other projects that might want to incentivise the use of their token). Then they would additionally receive rewards APR

Borrowers deposit collateral tokens in the LLAMMA, they can specify the number of bands, generally more bands translates to less losses in soft liquidation, but a wider liquidation range, less bands is the opposite, and additionally the LTV is bigger with less bands. They have to pay interests on the borrowed tokens. The borrow apy depends only on the utilisation rate of the market.

LLAMMA handles the soft liquidation mechanism.
Collateral is deposited in the 4 to 50 bands of the AMM. When the price of the collateral asset falls into the range of a band, the AMM starts selling collateral (soft liquidation), and if the price goes up again it starts buying collateral (de liquidation). This happens through adjusting the price of the collateral within the AMM so that arbitrage traders take the opportunity to buy or sell. Trading of collateral within the AMM leads to some losses during soft liquidation, that tend to get worse with higher volatility or lower liquidity. Liquidation is band based, meaning if several users have funds in the same band, they get liquidated together. If the price is within a certain bands, bands with higher prices are fully liquidated and only have the borrow token, and bands with lower prices are untouched and only have the collateral token, only the current band holds both the borrow token and the collateral token.
While losses are sustained during soft liquidation, this process has several benefits. It allows the position to mostly recover if the market recovers, which is impossible with hard liquidation. The progressive nature of soft liquidation makes borrowers less exposed to high volatility or oracle lag triggering the (hard) liquidation threshold, and losing their position without any recourse, while the prices might recover a moment after. Users also have the option to repay completely or partially their loan in soft liquidation, if they deem the market is not going their way. It should be noted though that hard liquidation can still happen when loan health drops below 0.

Traders, yield farmers, automated vaults that want the benefits of soft liquidation use llammalend. Many tokens are available to lend and borrow and some protocols offer incentives to do so.

3. Main contracts for yield farming

LLAMMA
Manages soft liquidation
Holds collateral and borrowable asset
Stores user's AMM shares

Controller
Central piece of a llama lend market
Create loan and manage existing positions
Tracks debt
External liquidations (hard liquidations)

Price oracle (curve oracle)
Time averaged oracle (EMA or SMA)
Fetches collateral price denominated against the borrowable token

Vault
Lenders send borrow token to receive supply vault shares.
Deposits assets into the Controller
Tracks the fees earned

Monetary Policy
Computes the borrow rate

CurveDAO
Admin of the monetary policy contract. Can change rates.
Admin of the factory of Controller. In the Controller, can change loan discount (used to define the LTV), liquidation discount (used to discount the collateral value when calculating the health for liquidation purposes in order to incentivize liquidators), the AMM fee, the admin fee, the monetary policy

Common actions

To lend
Call vault.deposit() or vault.mint(), the vault handles the process, with some interactions with the controller to get necessary values and update interest rates, and some interaction with the token contracts to transfer funds.

- Both call self._total_assets(), which is necessary to compute the amount of shares the lender will get in exchange for his assets
  - calls Controller.check_lock() to make sure no operation is happening in the controller at the same time,
  - calls borrowed_token.balanceOf() to get the balance of borrowable tokens of the Controller,
  - calls Controller.total_debt(), to get the total debt of the Controller,
    - calls AMM.get_rate_mul() to get the rate interest multiplier, which is used to compute the total debt of the controller, then we get the total assets as the sum of the total debt and the balance of borrow tokens.
- deposit() calls self._convert_to_shares(), to get the amount of shares to mint,
  - calls self._total_assets() to get the Controller assets, necessary to compute shares
- mint() calls self._convert_to_assets(), to get the number of assets required to mint a specific number of shares,
  - calls self._total_assets() to help determine amount of assets
- Both call borrowed_token.transferFrom(), to transfer the assets from the lender to the controller,
- Both call the controller.save_rate() to update the interest rate in the AMM
  - calls the monetary_policy.rate_write(), which returns the current rate of the lending market,
  - calls the AMM.set_rate() which updates the rate in the AMM.

To withdraw assets and redeem shares
Call vault.withdraw() or vault.redeem(), the vault handles the process, with some interactions with the controller to get necessary values and update interest rates, and some interaction with the token contracts to transfer funds.

- Both call self._total_assets(),
- withdraw() calls self._convert_to_shares(), to get the amount of shares to burn,
  - calls self._total_assets().
- redeem() calls self._convert_to_assets(), to get the number of assets to get from a specific number of shares
  - calls self._total_assets().
- Both call borrowed_token.transferFrom(), to transfer the assets from the controller to the lender,
- Both call the controller.save_rate()

To borrow / create loan
Call Controller.create_loan(), the controller handles the process with some underlying calls to the AMM to determine in which bands to deposit the collateral, and to deposit it, and transfers tokens via their contracts

- calls self._calculate_debt_n1() to get the lower band to deposit collateral into. This value is also used to compute the upper band, since the user specifies the desired number of bands.
  - calls AMM.active_band(), to get the band where the price of the collateral in the AMM is currently,
  - calls AMM.p_oracle_up(), to get the upper price bound of that band. Both values are used to compute the desired lower band.
  - calls AMM.can_skip_bands(), AMM.p_oracle_up() and AMM.price_oracle() to make sure the requested debt is not too high, otherwise the call reverts.
- calls AMM.get_rate_mul() to get the interest rate multiplier
- calls AMM.deposit_range() to deposit the collateral in the specified AMM bands (range between the specified lower and upper band)
- calls self.transferFrom(), which calls the collateral token transferFrom(), to transfer the collateral tokens from the user creating the loan, to the AMM.
- calls self.transfer(), which calls the borrow token transfer(), to transfer the borrowed tokens (the loan) to the user.
- calls self._save_rate(), to update the interest rate in the AMM

To create leverage using built-in functionality
Call Controller.create_loan_extended(),

- calls self.transfer(), calls borrowed_token.transfer() to send the debt to 1inch or Odos router
- calls self.execute_callback(),
  - calls the 1inch or Odos router to swap the loan for more collateral
  - calls the AMM to check that there was no interaction with the AMM (same active band, with same liquidity)
- calls self._create_loan()
  - calls self._calculate_debt_n1(), AMM.get_rate_mul() and AMM.deposit_range() to get the bands for this position and deposit collateral into them
  - calls self._save_rate(), to update the interest rate in the AMM
- calls self.transferFrom(), calls collateral_token.transferFrom() to transfer the collateral from the user creating the position to the AMM
- calls self.transferFrom(), calls collateral_token.transferFrom() to transfer the additional collateral from the 1inch or Odos router to the AMM

To repay loan
Call Controller.repay(), the controller handles the process with some underlying calls to the AMM to determine the status of the loan, withdraw assets if full repayment, and rebuild position (shift bands) if partial repayment, if the status of the loan allows, and transfers tokens via their contracts

- calls self._debt() to get the debt value, which calls the AMM.get_rate_mul() to compute the

current debt (which has interests)
If the debt is fully repaid:
calls AMM.withdraw() to withdraw user liquidity from the AMM
calls self.transferFrom(), to transfer borrow tokens from AMM to user
calls self.transferFrom(), to transfer collateral tokens from AMM to user
If partial repayment:
calls AMM.active_band_with_skip() to make sure the active band is less than the max
calls AMM.read_user_tick_numbers() gets the bands of the user's loan
If the loan is not in liquidation (which we can see by comparing the active band to the bands of the user's loan):
calls AMM.withdraw(), self.calculate_debt_n1(), AMM.deposit_range() to shift the user collateral into lower bands (further from liquidation)
if a user repays a loan for another user:
calls self._health() to make sure this doesn't make the loan unhealthy. Several AMM functions() are called in health() to get the values necessary to compute the health factor.
calls self.transferFrom, calls borrowed_token.transferFrom() to transfer (part of) the loan back from the user to the Controller
calls self._save_rate()

Adding collateral to a loan
Call Controller.add_collateral(), the controller handles the process with some underlying calls to the AMM to rebuild position (shift bands), and transfers tokens via their contracts
calls self. _add_collateral_borrow() to rebuild the position
calls self. _debt() to check if the loan exist and add debt if specified (in this case no)
calls AMM. read_user_tick_numbers() to get the current bands
calls AMM.withdraw() to withdraw liquidity from the current bands
calls self. _calculate_debt_n1() to recompute the liquidation range = the new bands after adding or removing (adding in this case) collateral. If debt is added or collateral is removed, the bands shift « up » (closer to current price, closer to liquidation). If collateral is added, the bands shift « down » (further from the current price, further from liquidation).
calls AMM.deposit_range() to put the liquidity back into the AMM in the new bands
calls self.transferFrom, calls collateral token transferFrom to transfer the additional collateral from the user to the AMM
calls self._save_rate()

Removing collateral from a loan
Call Controller.remove_collateral(), the controller handles the process with some underlying calls to the AMM to rebuild position (shift bands), and transfers tokens via their contracts
calls self._add_collateral_borrow() to rebuild the position with less collateral (band shift up)
calls self.transferFrom, calls collateral token transferFrom to transfer the additional collateral from the AMM to the user
calls self._save_rate()

Borrow more assets
Call Controller.borrow_more(), the controller handles the process with some underlying calls to the AMM to rebuild position (shift bands), and transfers tokens via their contracts
calls self. _add_collateral_borrow() to rebuild the position with more debt (band shift up)
calls self.transferFrom, which calls collateral token transferFrom to transfer the additional collateral from the user to the AMM
calls self.transfer, which calls borrow token transfer to transfer the additional borrow tokens from the Controller to the user
calls self._save_rate()

Liquidate
Call Controller.liquidate(), the controller handles the process with some underlying calls to the AMM to make sure the position can be liquidated and withdraw the liquidity, and transfers tokens via their contracts

- calls self.liquidate()
  - calls self._debt(), self._health() to make sure the loan is bad enough to liquidate
  - calls AMM.withdraw() to withdraw liquidity from the AMM
  - borrowed_token.transferFrom() and collateral_token.transferFrom() are called
    - several times to withdraw the tokens
  - calls Self._save_rate()

4. Dependencies and second order exposure

Let's take the wstETH - crvUSD market.

External dependencies

wstETH
stETH
ETH
Lido
Oracle
crvUSD
Curve
LLAMMA

Second order exposure

wstETH is the non rebasing version of stETH
- stETH. Exposure to all risks of stETH
  - Lido. Exposure to all risks of Lido governance. System changes, ecosystem (loss of trust)
  - stETH/ETH. Exposure to stETH/ETH (de)peg
  - Market volatility, liquidity, technical issues, hacks, regulation changes, slashing
    - ETH. Exposure to risks of ETH
    - cybersecurity threats, technical issues, volatility, liquidity, (de)centralisation, long term cryptographic risks

crvUSD is Curve's stable coin
- CurveDAO. Exposure to risks from Curve. Parameter changes, ecosystem (loss of trust)
- crvUSD/USD. Exposure to crvUSD/USD (de)peg.
- Exposure to all the assets that are provided as collateral for crvUSD
- Market volatility, liquidity, technical issues, hacks, regulation changes

Oracle
Exposure to oracle risks.
Technical bugs, manipulation, liquidity migration

LLAMMA
- borrow side
  - soft/hard liquidation
  - dynamic interest rates edge cases
- lend side
  - illiquidity
  - bad debt

6. AI Usage

I used ChatGPT. Absent professors, colleagues or friends to discuss problems, I like to brainstorm with chatgpt. I mostly use it to give me an overview of topics I don't know well before digging deeper, to find specific informations faster than digging through docs (e.g. how to do X in a certain programming language, what library can I use etc), and to give me another perspective which is sometimes useful in itself, and often useful for me to bounce back and make my own thinking move forward.
As such, I used it to do some preliminary research. Then I used it iteratively in the way I described. A caveat to this back and forth workflow is that AI always has a next step to do for you, and it's important to not get caught in a spiral and take a lot of steps back to focus on the actual problem to solve.
While I appreciate the comfort that it provides, I did not find AI exceptionally useful in this assignment. I think the reason is that, for me at least, it was a lot about when to simplify (and how), and when to go into details. Something I learned in security, you could have the perfect system, you still need to know exactly what you want from it, and any discrepancy can lead to significant errors. That's not something AI can give instantly. I completed the assignment before that section 6 existed so I didn't pay particular attention or kept track of any specific mistakes, more often it just simply did not give me exactly what I need.
I'm confident that all the submission is my own, AI just improved efficiency.